

Created for explanation of slide # 31 to 34 of Lecture – 7 (Query Optimization) uploaded at teams

1. Example Tables

We'll use three simple tables:

R(A, B) — Employees

A (emp_id) B (dept_id)

1	10	EMPID >> DEPTID >>> LOCID >>> LOCNAM
2	20	
3	10	
12	30	

S(B, C) — Departments

B (dept_id) C (location_id)

10	100
20	200
30	300

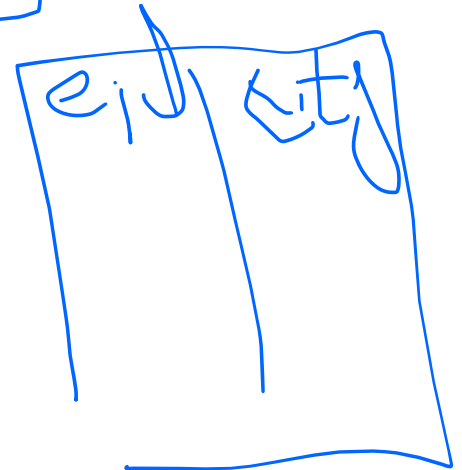
T(C, D) — Locations

C (location_id) D (city)

100	NY
200	LA
300	SF

2. SQL Query

```
SELECT R.A, T.D
FROM R, S, T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;
```



3. Step-by-Step Query Processing

Step 1: Join $R \bowtie S$ on $R.B = S.B$

A B C

1 10 100

2 20 200

3 10 100

12 30 300

Step 2: Join result with T on $S.C = T.C$

A B C D

1 10 100 NY

2 20 200 LA

3 10 100 NY

12 30 300 SF

Step 3: Apply Selection $\sigma (R.A < 10)$

A B C D

1 10 100 NY

2 20 200 LA

3 10 100 NY

Step 4: Projection $\pi (A, D)$

A D

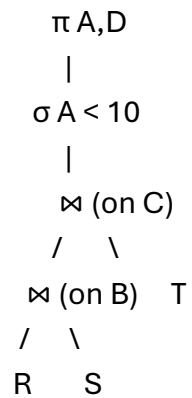
1 NY

2 LA

3 NY

4. Query Tree (Execution Plan)

Here's the equivalent **relational algebra tree**:

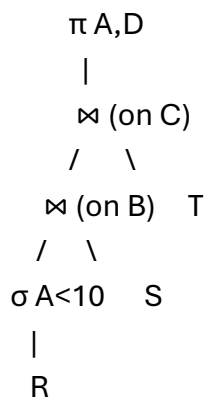


5. Optimized Query Plan (Important!)

The original plan is **not optimal**. We can push selection earlier:

Optimization:

Apply $\sigma(A < 10)$ before joins.



Why this is better:

- Reduces size of R early
 - Fewer rows participate in joins
 - Faster execution in real databases
-

6. Real-World Analogy

Think of it like:

- **R** = Employees
- **S** = Departments
- **T** = Office Locations

You're:

1. Filtering employees with ID < 10
2. Matching them to departments
3. Then to cities
4. Finally showing (employee_id, city)